

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES
(AUTONOMOUS)

(Permanent Affiliation by Andhra University & Approved by AICTE
Accredited by NBA & NAAC with A+ Grade)
Sangivalasa-531162, Bheemunipatnam Mandal,
Visakhapatnam District

A PROJECT WORK REPORT ON
RAILWAY RESERVATION SYSTEM

This project is submitted by

G.BINDU SHALINI	A25126510016
K.PRINCY	A25126510026
M.TEJASWINI	A25126510028
R.LAKSHMI CHARISHMA	A25126510046

Under the Guidance of

Dr B.MAHESH

Department of Computer Science and Engineering

in partial fulfillment of the requirements for the award of the
offered by

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES



Bachelor of Technology
in
COMPUTER SCIENCE AND ENGINEERING
2025-2029

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES
(AUTONOMOUS)

(Permanent Affiliation by Andhra University & Approved by AICTE)

Accredited by NBA & NAAC with A+ Grade)

Sangivalasa-531162, Bheemunipatnam Mandal,

Visakhapatnam District



CERTIFICATE

This is to certify that this project work entitled — “**RAILWAY RESERVATION SYSTEM**” is the bonafied record work done by **Ms. G.Bindu Shalini, Ms. K.Princy ,Ms. M.Tejaswini, Ms. R.Lakshmi Charishma** bearing Roll no:**A25126510016, A25126510026, A25126510028, A25126510046** of the second year , along with batch mates submitted on the partial fulfillment of the requirement for the award of B.tech in Computer Science and Engineering to the ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES. The results embodied in this project report have not been submitted to any other board/University or Institute for the award of diploma.

LECTURER IN CHARGE

HEAD OF THE DEPARTMENT

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who have contributed to the successful completion of this project. First and foremost, I extend my heartfelt thanks to my project guide, DR B.MAHESH, for their valuable guidance, continuous support, and encouragement throughout the development of this project. Their insightful feedback and expert advice were instrumental in shaping this work. I am also grateful to the faculty members of Computer Science and Engineering, ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES, for providing the necessary resources and a conducive environment for research and learning. A special thanks to my friends and classmates for their cooperation, constructive suggestions, and moral support during every stage of this project. Finally, I would like to thank my family for their unwavering love, patience, and constant motivation that kept me focused and determined. This project, titled “RAILWAY RESERVATION SYSTEM”, has been a great learning experience, helping me enhance my technical skills and practical understanding of the subject.

ABSTRACT

The Railway Reservation System is a computer-based application developed using C++ programming language to automate the process of train ticket booking and management. The main objective of this system is to provide an efficient and user-friendly platform for storing and retrieving train and passenger information. It reduces manual work, saves time, and improves accuracy in reservation operations. This project uses Object-Oriented Programming (OOP) concepts such as classes and objects to represent real-world entities like trains and passengers. It also implements file handling techniques to store train records and passenger booking details permanently in files, allowing data to be retained even after the program is closed.

PREFACE

Table of contents

Sl.No.	Chapter Title	Sub-Contents	Page No.
1	Introduction	Overview of the Project Purpose and Scope Existing System and Proposed System	1
2	Problem Definition	• Problem Statement • Objectives of the System	2
3	System Design	• System Architecture • Flowcharts • Algorithms Used	3-5
4	Implementation Details	• Program Structure • Language and Environment Used • Key Functional Modules • Code Snippets and Screenshots • File Handling Explanation	6-11
5	Results and Discussion	• Output Screens • Test Case Table (Input, Expected Output, Actual Output)	12-16

6	Conclusion	• Summary of Achievements • Limitations of the System • Future Enhancements	17
7	References	• Books Referred • Research Papers • Websites and Online Resources	18
8	Appendix	• Source Code Listings • Source Code	19-26

1.Introduction

Overview

The Railway Reservation System is designed to automate railway ticket booking operations. It allows users to check train availability, book tickets, cancel reservations, and display passenger details. The system stores data using file handling for permanent storage.

Purpose

The purpose of this project is to replace manual railway booking with a computerized system that improves speed, accuracy, and efficiency.

Scope

The scope of the project includes:-Train information management

- Passenger booking
- Ticket cancellation
- Seat availability tracking
- Data storage using files

Existing System

- Manual reservation system
- Time consuming process
- High chances of errors
- Difficult to maintain records
- No proper data security

Proposed system

- Computerized reservation system
- Fast booking and cancellation
- Accurate seat tracking
- Easy data storage
- User friendly interface

2.PROBLEM DEFINITION

PROBLEM DEFINITION

The traditional railway reservation system relies heavily on manual processes, making it inefficient and error-prone. Managing large amounts of passenger data and seat availability manually is difficult and time-consuming. There is a need for a computerized system that can handle booking, cancellation, and data management efficiently while ensuring accuracy and reliability.

OBJECTIVES

The main objective of the Railway Reservation System is to design and implement a computerized solution that simplifies and automates the railway ticket booking process.

1. Automation of Reservation Process

The system aims to eliminate manual work by automating ticket booking and cancellation. This reduces time consumption and increases efficiency.

2. Efficient Data Management

The system stores train and passenger details using file handling. This ensures that data is organized, easily accessible, and permanently stored.

3. Seat Availability Management

One of the key objectives is to track seat availability in real time. The system checks whether seats are available before confirming a booking.

4. User-Friendly Interface

The system is designed with a simple menu-driven interface so that users can easily interact with it without requiring technical knowledge.

5. Reduction of Errors

Manual systems are prone to errors such as duplicate entries or incorrect bookings. This system minimizes such errors through validation checks.

6. Implementation of OOP Concepts

The project demonstrates object-oriented programming concepts such as classes, objects, encapsulation.

3. SYSTEM DESIGN

System Design/Methodology

The Railway Reservation System is designed to simulate the process of booking and managing train tickets using programming concepts (e.g., C++/Java/Python). It enables users to search trains, reserve seats, cancel bookings, and check ticket details. The system maintains records using file handling or database techniques to store passenger details, train schedules, and reservation data. It ensures accurate and secure transactions through validation checks such as seat availability and user authentication. This project helps in understanding real-world railway booking logic and data management. It uses structured and object-oriented programming concepts for modular and efficient design. The system reduces manual work, improves accuracy, and provides fast access to reservation information. It can be extended with features like online payment integration, seat selection, and real-time train tracking.

Programming Concepts Used

1. File Handling/Database Management:

Stores all records such as:

- Train details (Train number, name, source, destination, timing)
- Passenger details (Name, age, gender)
- Reservation records (PNR number, seat number, class, status)

Transactions (booking/cancellation) are stored with:

- PNR Number
- Train Number
- Passenger Details
- Seat Status (Booked/Cancelled)
- Ensures persistent data storage and retrieval.

2. Glasses and Objects:

A structured format is used to represent system components:

Train Class: Stores train information like number, name, routes, and seat availability.

Passenger Class: Stores user details such as name, age, and contact info.

Reservation Class: Handles ticket booking, PNR generation, and status tracking.

This helps in modular design, easy updates, and efficient data handling.

3. Member Functions Used:

Train Management Functions:

- Add new train details
- Display available trains
- Check seat availability

Reservation (Booking) Function:

- Accept passenger details (name, age, gender)
- Select train and class
- Check seat availability
- Generate unique PNR number
- Allocate seat and confirm booking
- Store data in file/database
- Display message: "Ticket Booked Successfully"

Search Reservation Function:

- Input PNR number
- If found display booking details
- Else display "PNR not found"

Cancellation Function:

Input PNR number

If record exists:

- Cancel reservation
- Update seat availability
- Display "Ticket Cancelled Successfully"

Else:

Display "Invalid PNR Number"

Display Functions:

- Show all train details
- Show reservation records
- Show passenger details

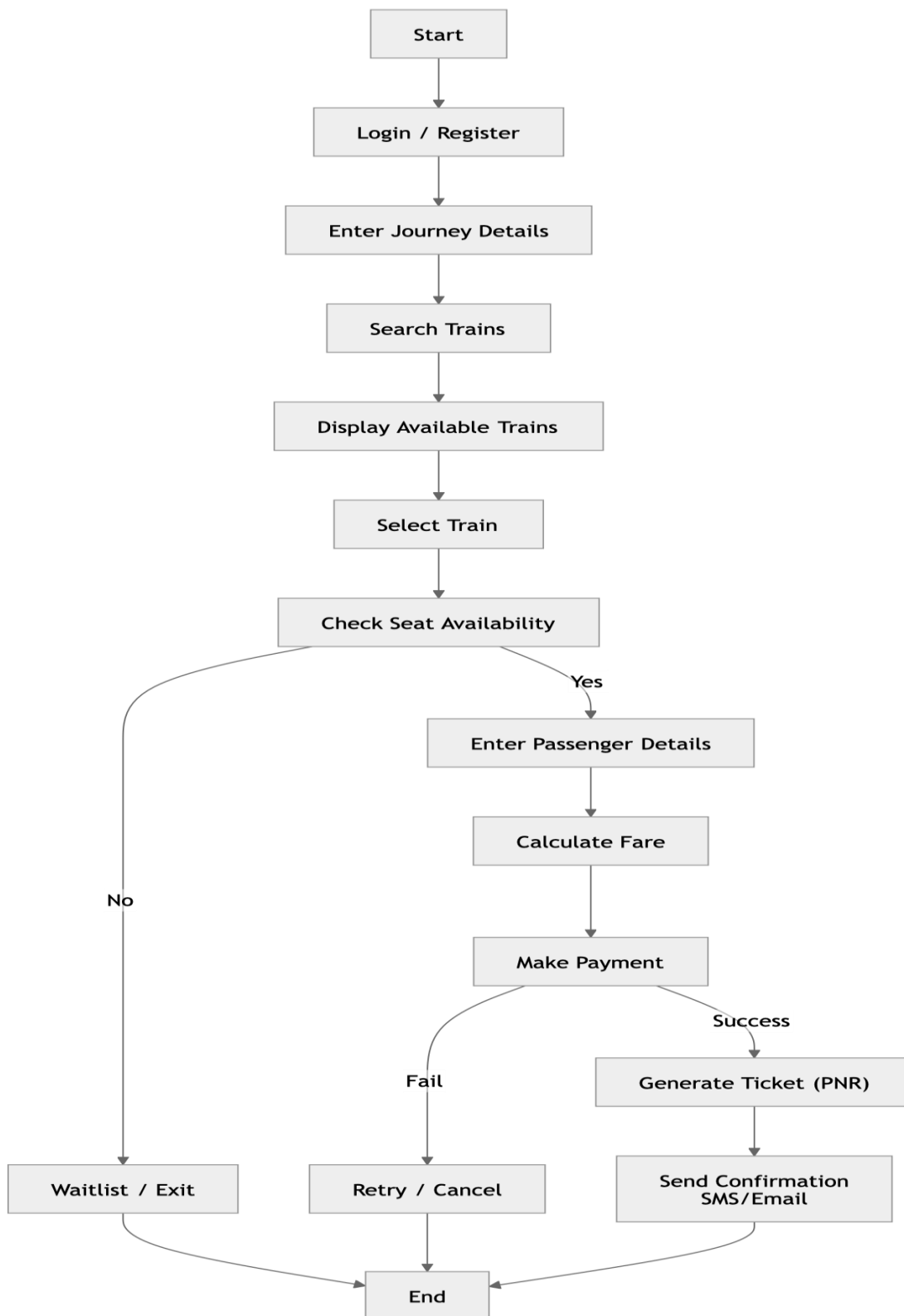
System Workflow (Summary)

1. User selects operation (Book/Cancel /Search/ View Trains)
2. System validates input
3. Performs requested operation
4. Updates records in file/database
5. Displays result to user

Advantages of the System

- Reduces manual booking errors
- Fast and efficient reservation process
- Secure and organized data management
- Easy to expand with advanced features

FLOW CHART:



4.IMPLEMANTATION DETAILS

Language and Environment Used

- Programming Language C++
- Libraries Used:
 - <iostream> : Input/Output operations
 - <fstream> : file handling
 - <cstdio> : File operations like remove() and rename()
- Data storage
Passenger details are stored using a text file (passenger.txt)
- **Each record contains:**
 - Adding train details
 - Viewing train list
 - Booking tickets
 - Cancelling tickets
 - Viewing booked tickets

Class design:

- Class name: Train

- Data members:

int	trainNo	:	Train	number
char	trainName[50]	:	Train	Name
int	totalSeats	:	No.of	seats
int	availableSeats	:	Available Seats	

1.add Train():

The addTrain() function is used to insert new train details into the Railway Reservation System. It is mainly designed for the Admin/Operator to add train information such as train number, train name, and total seats .The train details are stored in a file such as train.dat.The file is opened using **ofstream** in **append mode** so that new train records are added at the end of the file without deleting existing records. The addTrain() function is an essential part of the Railway Reservation System because it allows the administrator to add new train records and store them permanently using file handling.

```

void addTrain()
{
    cout<<"Enter Train Number: ";
    cin>>trainNo;

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout<<"Enter Train Name: ";
    cin.getline(trainName,50);

    cout<<"Enter Total Seats: ";
    cin>>seats;
}

void showTrain()
{
    cout<<"\nTrain No: "<<trainNo;
    cout<<"\nTrain Name: "<<trainName;
    cout<<"\nAvailable Seats: "<<seats<<endl;
}

```

2.Display train() :

The displayTrain() function in a Railway Reservation System is used to show the details of all trains available in the system. It reads the train information stored in a file and displays it on the screen in a proper format. It is an essential function in the Railway Reservation System because it provides a complete list of trains and supports other operations like searching trains and booking tickets. It show all train records stored in the file. It reads each train's details from the file and prints them on the screen.

```

void displayTrains()
{
    Train t;
    ifstream fin("train.dat",ios::binary);

    if(!fin)
    {
        cout<<"No Train Records Found\n";
        return;
    }

    while(fin.read((char*)&t,sizeof(t)))
    {
        t.showTrain();
    }
}

```

3.Book ticket() :

The bookTicket() function is used to reserve seats for a passenger in a Railway Reservation System. This is one of the most important functions because it performs the main reservation process. The function asks the user to enter the train number in which they want to book a ticket. This train number is used to search the train record in the train database. The train details are stored in a file such as train.dat. The function reads all train records one by one from the file. If the train is found, the system shows train information. The function verifies whether seats are available. If seats are available, the system asks passenger details. After successful booking, the system decreases the seat count. The passenger booking information is stored in another file. Finally, the system prints a message.

```
void bookTicket()
{
    cout<<"Enter PNR Number: ";
    cin>>pnr;

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout<<"Enter Passenger Name: ";
    cin.getline(name,50);

    // ✗ Removed train number input from here
}

void showTicket()
{
    cout<<"\nPNR: "<<pnr;
    cout<<"\nPassenger Name: "<<name;
    cout<<"\nTrain Number: "<<trainNo<<endl;
}
```

4.Cancel ticket() :

The cancelTicket() function is used to cancel an already booked ticket. When a passenger cancels a ticket. The system asks the user for details to identify the booking. The booking details are stored in a file. The program reads booking records one by one using a loop. If the passenger booking matches the entered details, then it means the ticket exists. If not found, it displays: "No Booking Found". To delete the cancelled booking, the program usually creates a temporary file. This is because in file handling, we cannot directly delete a specific record from a binary file easily. Once the ticket is cancelled, the system increases the train seat count. So the cancelled booking is removed permanently. Finally, the system prints: "Ticket Cancelled Successfully" and shows updated seats.

```

void cancelReservation()
{
    int pnrNo;
    char pname[50];
    Passenger p;

    ifstream fin("passenger.dat",ios::binary);
    ofstream fout("temp.dat",ios::binary);

    if(!fin)
    {
        cout<<"No Passenger Records Found\n";
        return;
    }

    cout<<"Enter PNR to Cancel: ";
    cin>>pnrNo;

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout<<"Enter Passenger Name: ";
    cin.getline(pname,50);

    int found=0;
    int trainNoToUpdate=-1;

    while(fin.read((char*)&p,sizeof(p)))
    {

```

```

        while(fin.read((char*)&p,sizeof(p)))
        {
            if(p.pnr==pnrNo && strcmp(p.name,pname)==0)
            {
                found=1;
                trainNoToUpdate=p.trainNo;
                cout<<"Reservation Cancelled\n";
            }
            else
            {
                fout.write((char*)&p,sizeof(p));
            }
        }

    fin.close();
    fout.close();

    remove("passenger.dat");
    rename("temp.dat","passenger.dat");

    // restore seat
    if(found)
    {
        Train t;
        fstream file("train.dat",ios::binary|ios::in|ios::out);

```

```

8
9
10 while(file.read((char*)&t,sizeof(t)))
11 {
12     if(t.trainNo==trainNoToUpdate)
13     {
14         t.seats++;
15         file.seekp(-sizeof(t),ios::cur);
16         file.write((char*)&t,sizeof(t));
17         break;
18     }
19 }
20 else
21 {
22     cout<<"No matching record found\n";
23 }
24 }
25

```

5.Display Passengers():

The displayPassengers() function is used to show the list of all passengers who booked tickets. It reads passenger booking records from the booking file and displays them on the screen. Passenger details are usually stored in a file. The function opens this file using ifstream in read mode. It uses a loop such as: while(file.read(...)). This loop continues until the file ends. After reading each passenger record, the function prints passenger information. This allows the admin to view all booked passengers. Once all records are read, the loop automatically stops when there are no more records in the file. This is important to avoid file corruption and free memory.


```

void displayPassengers()
{
    Passenger p;

    ifstream fin("passenger.dat",ios::binary);

    if(!fin)
    {
        cout<<"No Passenger Records Found\n";
        return;
    }

    while(fin.read((char*)&p,sizeof(p)))
    {
        p.showTicket();
    }
}

```

6.main() Function :

The main() function is the starting point of the Railway Reservation System program. It controls the complete flow of the program by displaying a menu and calling different functions like addTrain(), displayTrain(), bookTicket(), cancelTicket(), etc. The main() function shows a list of options to the user. The program takes the user choice. The switch(choice) statement is used to call the required function. The menu is shown repeatedly using a loop (do-while or while) so the user can perform multiple operations without closing the program. The loop continues until the user selects Exit option. When the user selects Exit, the loop stops and program ends.

```

int main()
{
    int ch;

    do
    {
        cout<<"\n1.Add Train";
        cout<<"\n2.Display Trains";
        cout<<"\n3.Book Ticket";
        cout<<"\n4.Cancel Ticket";
        cout<<"\n5.Display Passengers";
        cout<<"\n6.Exit";
        cout<<"\nEnter Choice: ";

        cin>>ch;

        if(cin.fail())
        {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout<<"Invalid input\n";
            continue;
        }
    }

```

```

    switch(ch)
    {
        case 1: addTrainRecord();
                break;
        case 2: displayTrains();
                break;
        case 3: bookReservation();
                break;
        case 4: cancelReservation();
                break;
        case 5: displayPassengers();
                break;
        case 6: cout<<"Exiting...\n";
                break;
        default: cout<<"Invalid Choice\n";
    }

    }while(ch!=6);

    return 0;
}

```

inout

Data Members Explanation:

The class Train contains the following data members:

- trainNo (int)
Stores the unique number of the train.
- trainName (char[50])
Stores the name of the train.
- seats (int)
Stores the total available seats in the train.

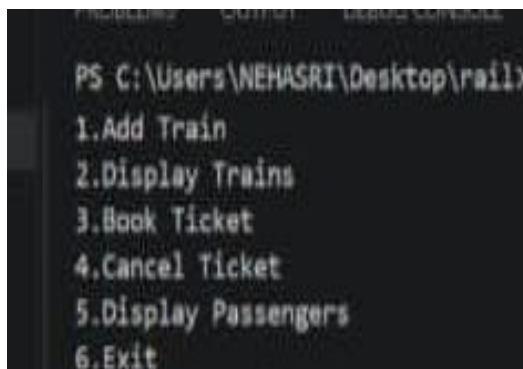
Outcomes:

A fully working railway reservation system featuring:

- Add Train – Allows admin to add new train details (train number, name, seats).
- Display Trains – Shows all available trains stored in the file.
- Book Ticket – Books a ticket for a passenger by checking seat availability and reducing seats.
- Cancel Ticket – Cancels an existing reservation and restores the seat count.
- Display Passengers – Displays all passenger booking details from the file.
- File Handling – Uses binary files (train.dat, passenger.dat) to store records permanently.
- Menu Driven Program – Provides an interactive menu in main() for user operations.

5.OUTPUT SCREENS

1.MAIN MENU SCREEN:



2.ADD TRAIN:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\NEHASRI\Desktop\rail> cd "c:\Users\NEHASRI\Desktop\rail"
1.Add Train
2.Display Trains
3.Book Ticket
4.Cancel Ticket
5.Display Passengers
6.Exit
Enter Choice: 1
Enter Train Number: 2345
Enter Train Name: abc
Enter Total Seats: 123

1.Add Train
2.Display Trains
3.Book Ticket
4.Cancel Ticket
5.Display Passengers
6.Exit
Enter Choice: 1
Enter Train Number: 1234
Enter Train Name: qwe
Enter Total Seats: 234

1.Add Train
2.Display Trains
3.Book Ticket
4.Cancel Ticket
5.Display Passengers
6.Exit
Enter Choice: 6
Exiting...
PS C:\Users\NEHASRI\Desktop\rail>
```

3.DISPLAY TRAINS:

```
← → rail
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\NEHASRI\Desktop\rail> cd "c:\Users\NEHASRI\Desktop\rail"
Enter Choice: 6
Exiting...
PS C:\Users\NEHASRI\Desktop\rail> cd "c:\Users\NEHASRI\Desktop\rail"

1.Add Train
2.Display Trains
3.Book Ticket
4.Cancel Ticket
5.Display Passengers
6.Exit
Enter Choice: 2

Train No: 2345
Train Name: abc
Available Seats: 123

Train No: 1234
Train Name: qwe
Available Seats: 234

1.Add Train
2.Display Trains
3.Book Ticket
4.Cancel Ticket
5.Display Passengers
6.Exit
Enter Choice: 6
Exiting...
PS C:\Users\NEHASRI\Desktop\rail>
```

4.BOOK TICKET: 4.1 If Trains are available

```
← → rail
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\NEHASRI\Desktop\rail> cd "c:\Users\NEHASRI\Desktop\rail\"
Enter Choice: 6
Exiting...
● PS C:\Users\NEHASRI\Desktop\rail> cd "c:\Users\NEHASRI\Desktop\rail\"

1.Add Train
2.Display Trains
3.Book Ticket
4.Cancel Ticket
5.Display Passengers
6.Exit
Enter Choice: 3
Enter Train Number for Booking: 2345
Enter PNR Number: 1234
Enter Passenger Name: xx
Ticket Booked Successfully

1.Add Train
2.Display Trains
3.Book Ticket
4.Cancel Ticket
5.Display Passengers
6.Exit
Enter Choice: 3
Enter Train Number for Booking: 2345
Enter PNR Number: 1234
Enter Passenger Name: qq
Ticket Booked Successfully

1.Add Train
2.Display Trains
3.Book Ticket
4.Cancel Ticket
5.Display Passengers
6.Exit
Enter Choice: 6
Exiting...
○ PS C:\Users\NEHASRI\Desktop\rail> 
```

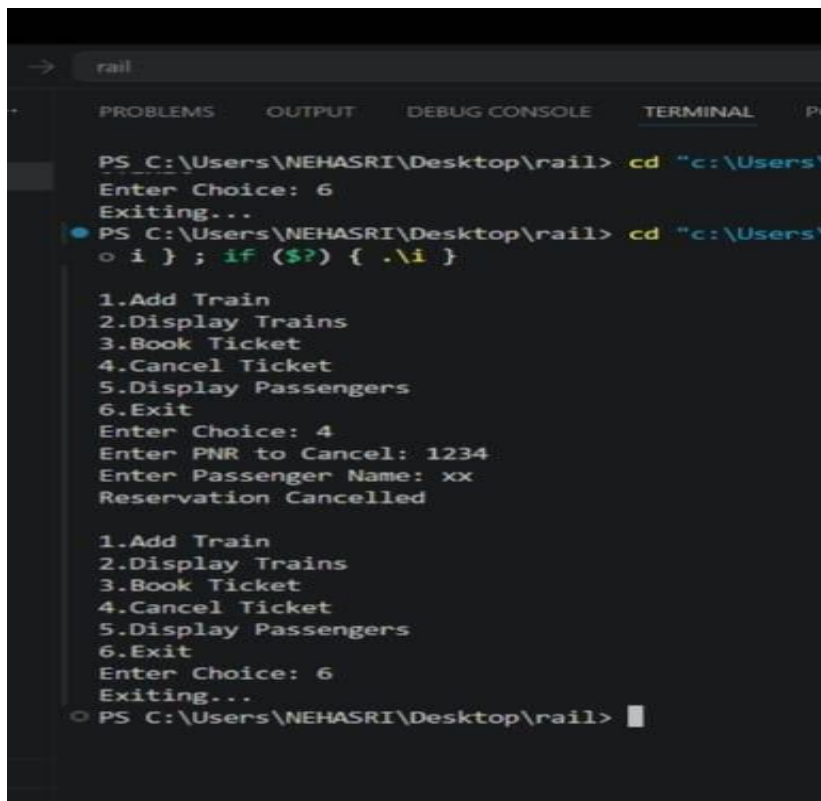
4.2 If Trains are not Available

```
PROBLEMS OUTPUT TERMINAL ... Code + v
PS C:\Users\NEHASRI\Desktop\rail> cd "c:\Users\NE
Exiting...
● PS C:\Users\NEHASRI\Desktop\rail> cd "c:\Users\NE
; if ($?) { g++ i.cpp -o i } ; if ($?) { .\i }

1.Add Train
2.Display Trains
3.Book Ticket
4.Cancel Ticket
5.Display Passengers
6.Exit
Enter Choice: 3
Enter Train Number for Booking: 67
Train Not Found

1.Add Train
2.Display Trains
3.Book Ticket
4.Cancel Ticket
5.Display Passengers
6.Exit
Enter Choice: 6
Exiting...
○ PS C:\Users\NEHASRI\Desktop\rail> 
```

5.CANCEL TICKET:

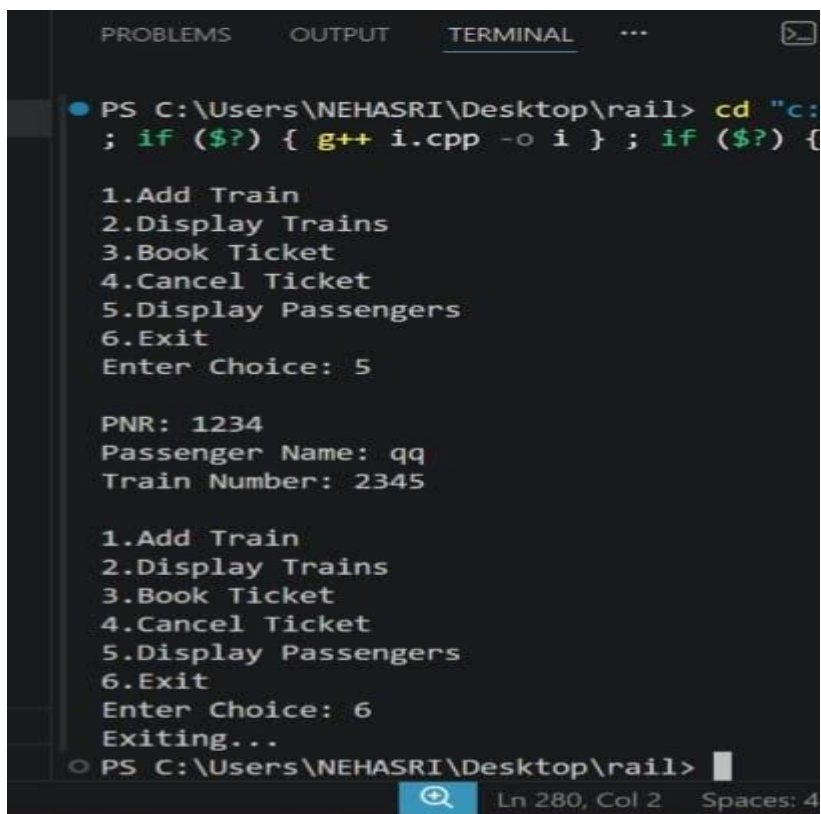


```
PS C:\Users\NEHASRI\Desktop\rail> cd "c:\Users\NEHASRI\Desktop\rail"
Enter Choice: 6
Exiting...
PS C:\Users\NEHASRI\Desktop\rail> cd "c:\Users\NEHASRI\Desktop\rail"
if ($?) { g++ i.cpp -o i } ; if ($?) { .\i }

1.Add Train
2.Display Trains
3.Book Ticket
4.Cancel Ticket
5.Display Passengers
6.Exit
Enter Choice: 4
Enter PNR to Cancel: 1234
Enter Passenger Name: xx
Reservation Cancelled

1.Add Train
2.Display Trains
3.Book Ticket
4.Cancel Ticket
5.Display Passengers
6.Exit
Enter Choice: 6
Exiting...
PS C:\Users\NEHASRI\Desktop\rail>
```

6.DISPLAY PASSENGERS:



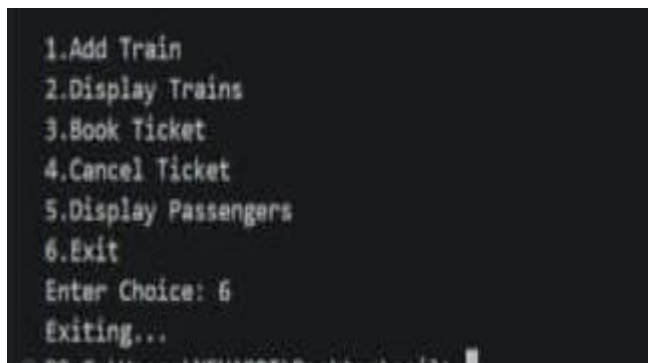
```
PS C:\Users\NEHASRI\Desktop\rail> cd "c:\Users\NEHASRI\Desktop\rail"
if ($?) { g++ i.cpp -o i } ; if ($?) { .\i }

1.Add Train
2.Display Trains
3.Book Ticket
4.Cancel Ticket
5.Display Passengers
6.Exit
Enter Choice: 5

PNR: 1234
Passenger Name: qq
Train Number: 2345

1.Add Train
2.Display Trains
3.Book Ticket
4.Cancel Ticket
5.Display Passengers
6.Exit
Enter Choice: 6
Exiting...
PS C:\Users\NEHASRI\Desktop\rail>
```

7.EXIT:



```
1.Add Train
2.Display Trains
3.Book Ticket
4.Cancel Ticket
5.Display Passengers
6.Exit
Enter Choice: 6
Exiting...
```

6.Conclusion

The Railway Reservation System effectively demonstrates how technology can streamline and modernize the process of booking train tickets. By replacing manual procedures with an automated system, it enables users to quickly check train schedules, seat availability, and fares, making the overall experience more convenient and time-efficient. The system reduces long queues and minimizes human errors, ensuring a smoother operation for both passengers and railway staff.

Another important outcome of the system is its ability to manage large volumes of data with accuracy and reliability. Through the use of a structured database, it maintains passenger details, booking records, and train information in an organized manner. This not only improves data retrieval speed but also enhances the overall efficiency of railway operations. The centralized system ensures consistency and better coordination across different modules.

Furthermore, the project highlights the importance of user-friendly design and accessibility. A well-designed interface allows users with basic technical knowledge to navigate the system easily, making it widely usable. The inclusion of features such as ticket cancellation, reservation status checking, and transaction handling adds to the system's practicality and usefulness in real-world scenarios.

In conclusion, the Railway Reservation System serves as a strong example of how digital solutions can improve public transportation services. While there is scope for further enhancement, the current system lays a solid foundation for building more advanced and scalable applications in the future. It emphasizes the role of automation in delivering faster, more reliable, and user-centric services.

Limitations

Despite its effectiveness, the system has certain limitations:

- It may not handle **high traffic loads efficiently** without proper scaling.
- The system might lack **real-time updates** if not connected to live railway data.
- Security features may be **basic**, making it vulnerable to unauthorized access if not enhanced.
- Limited payment integration options can restrict **user convenience**.

- The interface may not be fully optimized for **mobile devices or accessibility needs**.
- Dependency on database integrity means any failure could affect the entire system.

Future Extensions

The system can be further improved with the following enhancements:

- Integration with **real-time train tracking** and live seat availability.
- Implementation of **advanced security features** such as encryption and multi-factor authentication.
- Development of a **mobile application** for better accessibility.
 - Inclusion of **multiple payment gateways** (UPI, wallets, international cards).
 - Use of **AI-based recommendations** for suggesting routes, timings, or seat preferences.
 - Adding **multi-language support** to serve a wider audience.
 - Incorporation of **cloud-based infrastructure** for scalability and reliability.
 - Providing **notifications and alerts** via SMS or email for booking updates.

7.References

1. Indian Railways – Official Website: <https://indianrailways.gov.in>
Used for understanding real-world railway operations, schedules, and reservation processes.
2. IRCTC – Official Website: <https://www.irctc.co.in>
Reference for online ticket booking system design and functionality.
3. Database System Concepts by Abraham Silberschatz, Henry F. Korth, and S. Sudarshan
Used for database design principles and implementation.
4. Software Engineering by Ian Sommerville
Reference for system development lifecycle and project design.
5. Java: The Complete Reference by Herbert Schildt
Used for programming concepts if the project is implemented in Java.

8.APPENDIX

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <cstring>
```

```
#include <limits>
```

```
using namespace std;
```

```
class Train
```

```
{
```

```
public:
```



```

int trainNo;
char trainName[50];
int seats;

void addTrain()
{
    cout<<"Enter Train Number: ";
    cin>>trainNo;

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout<<"Enter Train Name: ";
    cin.getline(trainName,50);

    cout<<"Enter Total Seats: ";
    cin>>seats;
}

void showTrain()
{
    cout<<"\nTrain No: "<<trainNo;
    cout<<"\nTrain Name: "<<trainName;
    cout<<"\nAvailable Seats: "<<seats<<endl;
}

};

class Passenger
{
public:
    int pnr;
    char name[50];
    int trainNo;

```

```

void bookTicket()
{
    cout<<"Enter PNR Number: ";
    cin>>pnr;

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout<<"Enter Passenger Name: ";
    cin.getline(name,50);
}

void showTicket()
{
    cout<<"\nPNR: "<<pnr;
    cout<<"\nPassenger Name: "<<name;
    cout<<"\nTrain Number: "<<trainNo<<endl;
}

};

void addTrainRecord()
{
    Train t;
    ofstream fout("train.dat",ios::binary|ios::app);

    if(!fout)
    {
        cout<<"File Error\n";
        return;
    }

    t.addTrain();

```

```

        fout.write((char*)&t,sizeof(t));
    }

void displayTrains()
{
    Train t;
    ifstream fin("train.dat",ios::binary);

    if(!fin)
    {
        cout<<"No Train Records Found\n";
        return;
    }

    while(fin.read((char*)&t,sizeof(t)))
    {
        t.showTrain();
    }
}

void bookReservation()
{
    Passenger p;
    Train t;

    fstream file("train.dat",ios::binary|ios::in|ios::out);

    if(!file)
    {
        cout<<"Train file not found\n";
        return;
    }
}

```

```
int tno,found=0;
```

```
cout<<"Enter Train Number for Booking: ";
```

```
cin>>tno;
```

```
while(file.read((char*)&t,sizeof(t)))
```

```
{
```

```
    if(t.trainNo==tno)
```

```
    {
```

```
        found=1;
```

```
        if(t.seats>0)
```

```
        {
```

```
            p.bookTicket();
```

```
            p.trainNo = tno;
```

```
            ofstream fout("passenger.dat",ios::binary|ios::app);
```

```
            fout.write((char*)&p,sizeof(p));
```

```
            t.seats--;
```

```
            file.seekp(-sizeof(t),ios::cur);
```

```
            file.write((char*)&t,sizeof(t));
```

```
            cout<<"Ticket Booked Successfully\n";
```

```
        }
```

```
    else
```

```
    {
```

```
        cout<<"No Seats Available\n";
```

```
    }
```

```

        break;
    }
}

if(found==0)
    cout<<"Train Not Found\n";
}

void cancelReservation()
{
    int pnrNo;
    char pname[50];
    Passenger p;

    ifstream fin("passenger.dat",ios::binary);
    ofstream fout("temp.dat",ios::binary);

    if(!fin)
    {
        cout<<"No Passenger Records Found\n";
        return;
    }

    cout<<"Enter PNR to Cancel: ";
    cin>>pnrNo;

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout<<"Enter Passenger Name: ";
    cin.getline(pname,50);

```

```

int found=0;
int trainNoToUpdate=-1;

while(fin.read((char*)&p,sizeof(p)))
{
    if(p.pnr==pnrNo && strcmp(p.name,pname)==0)
    {
        found=1;
        trainNoToUpdate=p.trainNo;
        cout<<"Reservation Cancelled\n";
    }
    else
    {
        fout.write((char*)&p,sizeof(p));
    }
}

fin.close();
fout.close();

remove("passenger.dat");
rename("temp.dat","passenger.dat");

// restore seat
if(found)
{
    Train t;
    fstream file("train.dat",ios::binary|ios::in|ios::out);

    while(file.read((char*)&t,sizeof(t)))
    {
        if(t.trainNo==trainNoToUpdate)

```

```

        {
            t.seats++;
            file.seekp(-sizeof(t),ios::cur);
            file.write((char*)&t,sizeof(t));
            break;
        }
    }
}
else
{
    cout<<"No matching record found\n";
}
}

```

```

void displayPassengers()
{
    Passenger p;

    ifstream fin("passenger.dat",ios::binary);

    if(!fin)
    {
        cout<<"No Passenger Records Found\n";
        return;
    }

    while(fin.read((char*)&p,sizeof(p)))
    {
        p.showTicket();
    }
}

```

```
int main()
{
    int ch;

    do
    {
        cout<<"\n1.Add Train";
        cout<<"\n2.Display Trains";
        cout<<"\n3.Book Ticket";
        cout<<"\n4.Cancel Ticket";
        cout<<"\n5.Display Passengers";
        cout<<"\n6.Exit";
        cout<<"\nEnter Choice: ";

        cin>>ch;

        if(cin.fail())
        {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout<<"Invalid input\n";
            continue;
        }

        switch(ch)
        {
            case 1: addTrainRecord();
                    break;
            case 2: displayTrains();
                    break;
            case 3: bookReservation();
                    break;
```



```
        case 4: cancelReservation();
                break;
        case 5: displayPassengers();
                break;
        case 6: cout<<"Exiting...\n";
                break;
        default: cout<<"Invalid Choice\n";
    }

}while(ch!=6);

return 0;
}
```